

First-Order Logic

Ole C. Eidheim

September 24, 2026

Department of Computer Science

Outline

Introduction

First-order logic

Models

Example “by hand” proof and counterexample

Asking for variables

Example applications

Limitations of propositional logic

For example, expressing: Alice and Bob are students, and all students know arithmetic

Propositional logic:

$$KB = \{ \text{AlicelsStudent}, \\ \text{BobIsStudent}, \\ \text{AlicelsStudent} \Rightarrow \text{AliceKnowsArithmetic}, \\ \text{BobIsStudent} \Rightarrow \text{BobKnowsArithmetic} \}$$

First-order logic:

$$KB = \{ \text{Student}(\text{Alice}), \\ \text{Student}(\text{Bob}), \\ \forall x \text{ Student}(x) \Rightarrow \text{Knows}(x, \text{Arithmetic}) \}$$

Limitations of propositional logic

For example, expressing: Alice and Bob are students, and all students know arithmetic

Propositional logic:

$$KB = \{ \text{AlicelsStudent}, \\ \text{BobIsStudent}, \\ \text{AlicelsStudent} \Rightarrow \text{AliceKnowsArithmetic}, \\ \text{BobIsStudent} \Rightarrow \text{BobKnowsArithmetic} \}$$

Does Alice know arithmetic? Yes, since $KB \models \text{AliceKnowsArithmetic}$

First-order logic:

$$KB = \{ \text{Student}(\text{Alice}), \\ \text{Student}(\text{Bob}), \\ \forall x \text{ Student}(x) \Rightarrow \text{Knows}(x, \text{Arithmetic}) \}$$

Does Alice know arithmetic? Yes, since $KB \models \text{Knows}(\text{Alice}, \text{Arithmetic})$

Propositional and first-order logic in Prolog (not curriculum)

```
% Filename: students.prolog

consult(user). % Start of KB

% Propositional logic
aliceIsStudent.
bobIsStudent.
aliceKnowsArithmetic :- aliceIsStudent.
bobKnowsArithmetic  :- bobIsStudent.

% First-order logic
student(alice).
student(bob).
knows(X, arithmetic) :- student(X).

end_of_file. % End of KB

% Propositional logic
aliceKnowsArithmetic. % Does Alice know Arithmetic? Output: true
bobKnowsArithmetic.   % Does Bob know Arithmetic? Output: true

% First-order logic
knows(alice, arithmetic). % Does Alice know Arithmetic? Output: true
knows(bob, arithmetic).   % Does Bob know Arithmetic? Output: true

% Run for example with: swipl < students.prolog
```

Propositional vs first-order logic

Revisiting one of the examples from the propositional logic lecture

Propositional logic:

$$KB = \{ Rain, \\ Rain \Rightarrow Wet, \\ Wet \Rightarrow Slippery \}$$

First-order logic:

$$KB = \{ Is(Rain), \\ Is(Rain) \Rightarrow Is(Wet), \\ Is(Wet) \Rightarrow Is(Slippery) \}$$

Natural language demo (not curriculum)

First-order logic agent code example

Two example dialogs:

```
Hello! I am a first-order logic agent. Input 'kb' to show current knowledge base.  
> Alice does not speak english.  
I did not know that. The knowledge base has been updated.  
> Is Alice a student?  
I do not know.  
> All students speak english.  
I did not know that. The knowledge base has been updated.  
> Is Alice a student?  
No.
```

```
Hello! I am a first-order logic agent. Input 'kb' to show current knowledge base.  
> Alice is a student.  
I did not know that. The knowledge base has been updated.  
> Does Alice speak english?  
I do not know.  
> All students speak english.  
I did not know that. The knowledge base has been updated.  
> Does Alice speak english?  
Yes.
```

Outline

Introduction

First-order logic

Models

Example “by hand” proof and counterexample

Asking for variables

Example applications

First-order logic, definitions

- Term
 - Constant: specific object, e.g. *Alice*, *Bob*, *Arithmetic*
 - Variable: unspecified object, e.g. $x \in \{Alice, Bob, Arithmetic\}$
 - Function: mapping to an object, e.g. *Mother(Alice)* refers to *Carrie*
- Equality: two terms refer to the same object, e.g. *Bob* = *Bobby*
- Predicate: relation or property of terms, e.g. *Knows*, *Student*
- Sentence:
 - Atomic sentence: predicate or equality, applied to terms, that is *true* or *false*, e.g. $Knows(Alice, Arithmetic) \in \{true, false\}$
 - Complex sentence: connective or quantifier applied to sentences, e.g. $Knows(Alice, Arithmetic) \wedge Knows(Bob, Arithmetic)$

First-order logic, definitions

- Term
 - Constant: specific object, e.g. *Alice*, *Bob*, *Arithmetic*
 - Variable: unspecified object, e.g. $x \in \{Alice, Bob, Arithmetic\}$
 - Function: mapping to an object, e.g. *Mother*(*Alice*) refers to *Carrie*
- Equality: two terms refer to the same object, e.g. *Bob* = *Bobby*
- Predicate: relation or property of terms, e.g. *Knows*, *Student*
- Sentence:
 - Atomic sentence: predicate or equality, applied to terms, that is *true* or *false*, e.g. $Knows(Alice, Arithmetic) \in \{true, false\}$
 - Complex sentence: connective or quantifier applied to sentences, e.g. $Knows(Alice, Arithmetic) \wedge Knows(Bob, Arithmetic)$
- Quantifiers:
 - Universal quantification $\forall$: for all, e.g. Alice knows everything:
 $\forall x \text{ Knows}(Alice, x) \equiv Knows(Alice, Arithmetic) \wedge Knows(Alice, English) \wedge \dots$
 - Existential quantification $\exists$: there exists, e.g. Alice knows something:
 $\exists x \text{ Knows}(Alice, x) \equiv Knows(Alice, Arithmetic) \vee Knows(Alice, English) \vee \dots$

Functions, continued

For example, expressing: *Alice's mother is Carrie, Bob's mother is Denise, and all mothers love their children*

Without functions:

$$KB = \{ \text{Mother}(\text{Alice}, \text{Carrie}), \\ \text{Mother}(\text{Bob}, \text{Denise}), \\ \forall x \forall y \text{ Mother}(x, y) \Rightarrow \text{Loves}(y, x) \}$$

With a *Mother* function:

$$KB = \{ \text{Mother}(\text{Alice}) = \text{Carrie}, \\ \text{Mother}(\text{Bob}) = \text{Denise}, \\ \forall x \text{ Loves}(\text{Mother}(x), x) \}$$

Nested quantifiers

Consecutive quantifiers of the **same type**, e.g. $\forall x \forall y$, can be written as one quantifier with several variables:

$$\forall x, y \text{ Brother}(x, y) \Rightarrow \text{Sibling}(x, y)$$

all brothers are siblings

Nested quantifiers

Consecutive quantifiers of the **same type**, e.g. $\forall x \forall y$, can be written as one quantifier with several variables:

$$\forall x, y \text{ Brother}(x, y) \Rightarrow \text{Sibling}(x, y)$$

all brothers are siblings

Order of consecutive quantifiers of **different types** matters:

$$\forall x \exists y \text{ Loves}(x, y) \quad \neq \quad \exists y \forall x \text{ Loves}(x, y)$$

everyone loves someone $\neq$ there exists someone who is loved by everyone

Connections between $\forall$ and $\exists$

De Morgan rules for quantified sentences

Quantified sentences

$$\neg \forall x P(x) \equiv \exists x \neg P(x)$$

$$\neg \exists x P(x) \equiv \forall x \neg P(x)$$

$$\forall x P(x) \equiv \neg \exists x \neg P(x)$$

$$\exists x P(x) \equiv \neg \forall x \neg P(x)$$

Unquantified sentences

$$\neg(P(A) \wedge P(B)) \equiv \neg P(A) \vee \neg P(B)$$

$$\neg(P(A) \vee P(B)) \equiv \neg P(A) \wedge \neg P(B)$$

$$P(A) \wedge P(B) \equiv \neg(\neg P(A) \vee \neg P(B))$$

$$P(A) \vee P(B) \equiv \neg(\neg P(A) \wedge \neg P(B))$$

Example:

$$\forall x Likes(x, IceCream) \equiv \neg \exists x \neg Likes(x, IceCream)$$

$$\text{everyone likes ice cream} \equiv \text{there is no one who does not like ice cream}$$

Natural language quantifiers

Universal quantification $\forall$

Every student knows arithmetic:

Natural language quantifiers

Universal quantification $\forall$

Every student knows arithmetic:

$$\forall x \textit{ Student}(x) \wedge \textit{ Knows}(x, \textit{ Arithmetic})$$

Natural language quantifiers

Universal quantification $\forall$

Every student knows arithmetic:

$$\forall x \text{ Student}(x) \Rightarrow \text{Knows}(x, \text{Arithmetic})$$

Natural language quantifiers

Universal quantification $\forall$

Every student knows arithmetic:

$$\forall x \text{ Student}(x) \Rightarrow \text{Knows}(x, \text{Arithmetic})$$

Existential quantification $\exists$

Some students knows arithmetic:

Natural language quantifiers

Universal quantification $\forall$

Every student knows arithmetic:

$$\forall x \text{ Student}(x) \Rightarrow \text{Knows}(x, \text{Arithmetic})$$

Existential quantification $\exists$

Some students knows arithmetic:

$$\exists x \text{ Student}(x) \Rightarrow \text{Knows}(x, \text{Arithmetic})$$

Natural language quantifiers

Universal quantification $\forall$

Every student knows arithmetic:

$$\forall x \text{ Student}(x) \Rightarrow \text{Knows}(x, \text{Arithmetic})$$

Existential quantification $\exists$

Some students knows arithmetic:

$$\exists x \text{ Student}(x) \wedge \text{Knows}(x, \text{Arithmetic})$$

Natural language quantifiers, continued

There is some course that every student takes:

Natural language quantifiers, continued

There is some course that every student takes:

$$\exists y \text{ Course}(y) \wedge (\forall x \text{ Student}(x) \Rightarrow \text{Takes}(x, y))$$

Natural language quantifiers, continued

There is some course that every student takes:

$$\exists y \text{ Course}(y) \wedge (\forall x \text{ Student}(x) \Rightarrow \text{Takes}(x, y))$$

Every even integer greater than 2 is the sum of two primes (Goldbach's conjecture):

Natural language quantifiers, continued

There is some course that every student takes:

$$\exists y \text{ Course}(y) \wedge (\forall x \text{ Student}(x) \Rightarrow \text{Takes}(x, y))$$

Every even integer greater than 2 is the sum of two primes (Goldbach's conjecture):

$$\forall x \text{ Even}(x) \wedge \text{Greater}(x, 2) \Rightarrow (\exists y, z \text{ Equals}(x, \text{Sum}(y, z)) \wedge \text{Prime}(y) \wedge \text{Prime}(z))$$

Natural language quantifiers, continued

There is some course that every student takes:

$$\exists y \text{ Course}(y) \wedge (\forall x \text{ Student}(x) \Rightarrow \text{Takes}(x, y))$$

Every even integer greater than 2 is the sum of two primes (Goldbach's conjecture):

$$\forall x \text{ Even}(x) \wedge \text{Greater}(x, 2) \Rightarrow (\exists y, z \text{ Equals}(x, \text{Sum}(y, z)) \wedge \text{Prime}(y) \wedge \text{Prime}(z))$$

If a student takes a course and the course covers a concept, then the student knows that concept:

Natural language quantifiers, continued

There is some course that every student takes:

$$\exists y \text{ Course}(y) \wedge (\forall x \text{ Student}(x) \Rightarrow \text{Takes}(x, y))$$

Every even integer greater than 2 is the sum of two primes (Goldbach's conjecture):

$$\forall x \text{ Even}(x) \wedge \text{Greater}(x, 2) \Rightarrow (\exists y, z \text{ Equals}(x, \text{Sum}(y, z)) \wedge \text{Prime}(y) \wedge \text{Prime}(z))$$

If a student takes a course and the course covers a concept, then the student knows that concept:

$$\forall x, y, z \text{ Student}(x) \wedge \text{Course}(y) \wedge \text{Concept}(z) \wedge \text{Takes}(x, y) \wedge \text{Covers}(y, z) \Rightarrow \text{Knows}(x, z)$$

Natural language quantifiers, equality

Bob has at least two brothers:

$$\exists x, y \text{ Brother}(\text{Bob}, x) \wedge \text{Brother}(\text{Bob}, y)$$

Natural language quantifiers, equality

Bob has at least two brothers:

$$\exists x, y \text{ Brother}(\text{Bob}, x) \wedge \text{Brother}(\text{Bob}, y) \wedge \neg(x = y)$$

Outline

Introduction

First-order logic

Models

Example “by hand” proof and counterexample

Asking for variables

Example applications

Models in propositional and first-order logic, database semantics

Review: models in propositional logic

Symbol assignments, e.g. $\{AlicelsStudent = true, BobIsStudent = false,$
 $AliceKnowsArithmetic = true, BobKnowsArithmetic = false\}$

Models in propositional and first-order logic, database semantics

Review: models in propositional logic

Symbol assignments, e.g. $\{AliceIsStudent = true, BobIsStudent = false, AliceKnowsArithmetic = true, BobKnowsArithmetic = false\}$

Models in first-order logic

In the simplest case, **database semantics**: predicate assignments, e.g.:

$\{Student = \{Alice = true, else = false\}, Knows = \{(Alice, Arithmetic) = true, else = false\}\}$

- $Student(Alice)$ is *true* since *Alice* is assigned *true* in the *Student* set
- $Student(Bob)$ is *false* since all except *Alice* is assigned *false* in the *Student* set
- $Knows(Alice, Arithmetic)$ is *true* since $(Alice, Arithmetic)$ is assigned *true* in the *Knows* set

Models in propositional and first-order logic, database semantics

Review: models in propositional logic

Symbol assignments, e.g. $\{AliceIsStudent = true, BobIsStudent = false, AliceKnowsArithmetic = true, BobKnowsArithmetic = false\}$

Models in first-order logic

In the simplest case, **database semantics**: predicate assignments, e.g.:

$\{Student = \{Alice = true, else = false\}, Knows = \{(Alice, Arithmetic) = true, else = false\}\}$

- $Student(Alice)$ is *true* since *Alice* is assigned *true* in the *Student* set
- $Student(Bob)$ is *false* since all except *Alice* is assigned *false* in the *Student* set
- $Knows(Alice, Arithmetic)$ is *true* since $(Alice, Arithmetic)$ is assigned *true* in the *Knows* set

This model satisfies e.g.:

$KB = \{Student(Alice), \neg Student(Bob), \forall x \ Student(x) \Rightarrow Knows(x, Arithmetic)\}$

Models in first-order logic, objects in standard semantics

In **standard semantics**, separate objects are used in the predicate assignments, where:

- Two or more constants can be assigned to the same object, e.g.:

Constant assignments: $\{Bob = B, Bobby = B\}$

Predicate assignments: $\{Student = \{else = true\}\}$

Models in first-order logic, objects in standard semantics

In **standard semantics**, separate objects are used in the predicate assignments, where:

- Two or more constants can be assigned to the same object, e.g.:
Constant assignments: $\{Bob = B, Bobby = B\}$
Predicate assignments: $\{Student = \{else = true\}\}$
- There may exist objects that are not assigned to by any constant, e.g.:
Constant assignments: $\{Bob = A, Bobby = B\}$
Predicate assignments: $\{Student = \{C = false, else = true\}\}$

The above models satisfy e.g. $KB = \{Student(Bob), Student(Bobby)\}$

Models in first-order logic, objects in standard semantics

There can be many, possibly surprising, models that satisfy a given KB, e.g.:

Constant assignments: $\{Alice = A, Carrie = A, Denise = A\}$

Predicate assignments: $\{Mother = \{else = true\}, Loves = \{else = true\}\}$

satisfies $KB = \{Mother(Alice, Carrie),$
 $Mother(Carrie, Denise),$
 $\forall x, y \text{ } Mother(x, y) \Rightarrow Loves(y, x)\}$

Models in first-order logic, objects in standard semantics

There can be many, possibly surprising, models that satisfy a given KB, e.g.:

Constant assignments: $\{Alice = A, Carrie = A, Denise = A\}$

Predicate assignments: $\{Mother = \{else = true\}, Loves = \{else = true\}\}$

satisfies $KB = \{Mother(Alice, Carrie),$
 $Mother(Carrie, Denise),$
 $\forall x, y \text{ } Mother(x, y) \Rightarrow Loves(y, x)\}$

But we can still decide entailment, e.g.:

$$KB \models Loves(Carrie, Alice)$$
$$KB \not\models Loves(Alice, Carrie)$$
$$KB \models Loves(Denise, Carrie)$$
$$KB \not\models Loves(Carrie, Denise)$$

Models in first-order logic, functions in standard semantics

Example model with function assignments:

Constant assignments: $\{Alice = A, Carrie = C, Denise = D\}$

Predicate assignments: $\{Loves = \{else = true\}\}$

Function assignments: $\{Mother = \{A = C, else = D\}\}$

that satisfies e.g. $KB = \{Mother(Alice) = Carrie,$
 $Mother(Carrie) = Denise,$
 $\forall x Loves(Mother(x), x) \}$

Models in first-order logic, functions in standard semantics

Example model with function assignments:

Constant assignments: $\{Alice = A, Carrie = C, Denise = D\}$

Predicate assignments: $\{Loves = \{else = true\}\}$

Function assignments: $\{Mother = \{A = C, else = D\}\}$

that satisfies e.g. $KB = \{Mother(Alice) = Carrie,$
 $Mother(Carrie) = Denise,$
 $\forall x Loves(Mother(x), x) \}$

Again, we can still decide entailment, e.g.:

$$KB \models Loves(Carrie, Alice)$$
$$KB \not\models Loves(Alice, Carrie)$$
$$KB \models Loves(Denise, Carrie)$$
$$KB \not\models Loves(Carrie, Denise)$$

Models in first-order logic, infinite standard semantics models

The following knowledge base includes the part of the Peano axioms that define an infinite sequence of distinct objects:

$$\begin{aligned} KB = \{ & \text{NatNum}(0), \\ & \forall x \text{ NatNum}(x) \Rightarrow \text{NatNum}(\text{Successor}(x)), \\ & \forall x \neg(\text{Successor}(x) = 0), \\ & \forall x, y \neg(x = y) \Rightarrow \neg(\text{Successor}(x) = \text{Successor}(y)) \} \end{aligned}$$

This can be interpreted as e.g. natural numbers: $0, \text{Successor}(0), \text{Successor}(\text{Successor}(0)), \dots$

An example **standard semantics** model that satisfies the above KB:

Constant assignments: $\{0 = A\}$

Predicate assignments: $\{\text{NatNum} = \{\text{else} = \text{true}\}\}$

Function assignments: $\{\text{Successor} = \{A = C, C = B, B = D, \dots\}\}$

Outline

Introduction

First-order logic

Models

Example “by hand” proof and counterexample

Asking for variables

Example applications

Example “by hand” proof and counterexample (not curriculum)

Is this equivalence correct?

$$\forall x P(x) \wedge Q(x) \equiv (\forall x P(x)) \wedge (\forall x Q(x))$$

Example “by hand” proof and counterexample (not curriculum)

Is this equivalence correct?

$$\forall x P(x) \wedge Q(x) \equiv (\forall x P(x)) \wedge (\forall x Q(x))$$

Proof of $\frac{\forall x P(x) \wedge Q(x)}{(\forall x P(x)) \wedge (\forall x Q(x))}$ “by hand”:

1	$\forall x P(x) \wedge Q(x)$	Assumption
2	a	let a be an arbitrary object in x 's domain
3	$P(a) \wedge Q(a)$	$\forall$ elimination 1
4	$P(a)$	$\wedge$ elimination 3
5	$Q(a)$	$\wedge$ elimination 3
6	$\forall x P(x)$	$\forall$ introduction 2-4
7	$\forall x Q(x)$	$\forall$ introduction 2-5
8	$(\forall x P(x)) \wedge (\forall x Q(x))$	$\wedge$ introduction 6-7

Must also prove the other direction (next slide)

Example “by hand” proof and counterexample (not curriculum)

Is this equivalence correct?

$$\forall x P(x) \wedge Q(x) \equiv (\forall x P(x)) \wedge (\forall x Q(x))$$

Proof of $\frac{(\forall x P(x)) \wedge (\forall x Q(x))}{\forall x P(x) \wedge Q(x)}$ “by hand”:

- | | | |
|---|--|---|
| 1 | $(\forall x P(x)) \wedge (\forall x Q(x))$ | Assumption |
| 2 | $\forall x P(x)$ | $\wedge$ elimination 1 |
| 3 | $\forall x Q(x)$ | $\wedge$ elimination 1 |
| 4 | a | let a be an arbitrary object in x 's domain |
| 5 | $P(a)$ | $\forall$ elimination 2 |
| 6 | $Q(a)$ | $\forall$ elimination 3 |
| 7 | $P(a) \wedge Q(a)$ | $\wedge$ introduction 5-6 |
| 8 | $\forall x P(x) \wedge Q(x)$ | $\forall$ introduction 4-7 |

Answer: yes

Example “by hand” proof and counterexample (not curriculum)

Is this equivalence correct?

$$\forall x P(x) \vee Q(x) \equiv (\forall x P(x)) \vee (\forall x Q(x))$$

Example “by hand” proof and counterexample (not curriculum)

Is this equivalence correct?

$$\forall x P(x) \vee Q(x) \equiv (\forall x P(x)) \vee (\forall x Q(x))$$

Counterexample:

Predicate assignments: $\{P = \{A = \text{true}, B = \text{false}\},$
 $Q = \{A = \text{false}, B = \text{true}\} \}$

Answer: no

Outline

Introduction

First-order logic

Models

Example “by hand” proof and counterexample

Asking for variables

Example applications

Asking for variables

Let $KB = \{Student(Alice),$
 $Student(Bob),$
 $\forall x \ Student(x) \Rightarrow Knows(x, Arithmetic)\}$

Who knows arithmetic, i.e. which values of x satisfy $KB \models Knows(x, Arithmetic)$?

Asking for variables

Let $KB = \{Student(Alice),$
 $Student(Bob),$
 $\forall x Student(x) \Rightarrow Knows(x, Arithmetic)\}$

Who knows arithmetic, i.e. which values of x satisfy $KB \models Knows(x, Arithmetic)$?

Answer: *Alice* and *Bob*, since:

$$KB \models Knows(Alice, Arithmetic)$$

$$KB \models Knows(Bob, Arithmetic)$$

Asking for variables, Prolog example (not curriculum)

```
consult(user). % Start of KB

student(alice).
student(bob).
knows(X, arithmetic) :- student(X).

end_of_file. % End of KB

knows(X, arithmetic), write(X), nl, false. % Output: alice, bob
```

Asking for variables, Z3 example (not curriculum)

```
from z3 import *

Object = DeclareSort("Object")

# Constants
Alice, Bob, Arithmetic = Consts("Alice Bob Arithmetic", Object)

# Predicates
Student = Function("Student", Object, BoolSort())
Knows = Function("Knows", Object, Object, BoolSort())
Answer = Function("Answer", Object, BoolSort())

# Variables
x = Const("x", Object)

s = Solver()

# KB
s.add(Student(Alice))
s.add(Student(Bob))
s.add(ForAll([x], Implies(Student(x), Knows(x, Arithmetic))))

s.add(ForAll([x], Knows(x, Arithmetic) == Answer(x))) # Ask for variables

print(s.check()) # sat
print(s.model())
```

```
# Output (subject to change):
# sat
# [Alice = Object!val!0,
#   Bob = Object!val!1,
#   Arithmetic = Object!val!2,
#   Student = [else -> True],
#   Knows = [else -> True],
#   Answer = [
#     Object!val!0 -> True,
#     Object!val!1 -> True,
#     else -> Knows(Var(0), Object!val!2)]]
```

Outline

Introduction

First-order logic

Models

Example “by hand” proof and counterexample

Asking for variables

Example applications

Prove: $\sqrt{2}$ is an irrational number (not curriculum)

$\sqrt{2}$ is an irrational number if $\neg(\sqrt{2} = \frac{A}{B})$, i.e. $\neg(A^2 = 2B^2)$, where $A, B \in \mathbb{Z}^+$

$KB = \{\forall x \text{ Multiply}(x, 1) = x,$	identity
$\forall x, y \text{ Multiply}(x, y) = \text{Multiply}(y, x),$	commutativity
$\forall x, y, z \text{ Multiply}(x, \text{Multiply}(y, z)) = \text{Multiply}(\text{Multiply}(x, y), z),$	associativity
$\forall x, y, z \text{ Multiply}(x, y) = \text{Multiply}(x, z) \Rightarrow y = z,$	cancellation, $x \neq 0$
$\forall x, y \text{ Divides}(x, y) \Leftrightarrow (\exists z \text{ Multiply}(x, z) = y),$	$\text{Divides } x y$ definition
$\forall x \text{ Divides}(2, \text{Multiply}(x, x)) \Rightarrow \text{Divides}(2, x),$	$2 x^2 \Rightarrow 2 x$
$\forall x \neg(x = 1) \Rightarrow \neg(\text{Divides}(x, A) \wedge \text{Divides}(x, B)),$	$\frac{A}{B}$ in lowest terms
$\neg(2 = 1)$	}

Answer: yes, since $M(KB \cup \{\neg\neg(\text{Multiply}(A, A) = \text{Multiply}(2, \text{Multiply}(B, B)))\}) = \emptyset$

Prove: if x^2 is even, then x is even (not curriculum)

```
from z3 import * # type: ignore

set_param("proof", True)

x = Int('x')

s = Solver()

def Even(x):
    return x % 2 == 0

s.add(Not(Implies(Even(x * x), Even(x)))) # Prove that if x^2 is even,
                                           # then x is even

result = s.check()

print(result) # unsat
if result == sat: # A model was found that satisfies the assumptions
    print(s.model()) # Show model
elif result == unsat: # A contradiction was found
    print(s.proof()) # Show proof
```

Prove: the sum of two odd numbers is an even number (not curriculum)

```
from z3 import * # type: ignore

set_param("proof", True)

x, y, k = Ints('x y k')

s = Solver()

def Odd(x):
    return Exists([k], x == 2 * k + 1)

def Even(x):
    return Exists([k], x == 2 * k)

s.add(Odd(x)) # x is an odd number
s.add(Odd(y)) # y is an odd number

s.add(Not(Even(x + y))) # Prove that x + y is an even number

result = s.check()

print(result) # unsat
if result == sat: # A model was found that satisfies the assumptions
    print(s.model()) # Show model
elif result == unsat: # A contradiction was found
    print(s.proof()) # Show proof
```

Satisfiability Modulo Theories (SMT) (not curriculum)

SMT (SAT + theory solvers), such as the [Z3 Theorem Prover](#), can also be used on problems like:

$$y = x + 1, \quad x \geq 0, \quad y < 1 \vee y > 2, \quad x, y \in \mathbb{R}$$

where logic is combined with e.g. [theory of arithmetic](#) to solve linear programming problems

Alive2: verification of LLVM code and transformations (not curriculum)

- Used to e.g. check if optimization rules are correct
- Have found more than 100 bugs in LLVM (not counting bugs found by Alive1)
- Have found more than 20 bugs in Z3
- Try it online: [alive-tv](#)
- Uses the [Z3 Theorem Prover](#)

Prove: no integer overflow/underflow (not curriculum)

```
from z3 import * # type: ignore

set_param("proof", True)

n = BitVec('n', 32) # Uses theory of bit-vectors

s = Solver()

s.add(Not(Or(n < 0, n >= 46341)))

s.add(
    Not(And(BVMulNoOverflow(n, n + 1, True), BVMulNoUnderflow(n, n + 1)))
) # Prove no multiplication overflow/underflow
# For non-negative ints a and b, no overflow if: Or(a == 0, b <= 0x7fffffff / a):
# s.add(Not(Or(n == 0, n + 1 <= 0x7fffffff / n)))

result = s.check()
print(result) # unsat
if result == sat: # A model was found that satisfies the assumptions
    print(s.model()) # Show model
elif result == unsat: # A contradiction was found
    print(s.proof()) # Show proof
```

```
int triangular_number(int n) {
    if (n < 0 || n >= 46341)
        return -1;

    return (n * (n + 1)) / 2;
}
```

Prove: no out-of-bounds reads (not curriculum)

```
from z3 import * # type: ignore

set_param("proof", True)

input_size = BitVec('input_size', 64)
result_size = BitVec('result_size', 64)
i = BitVec('i', 64)
```

```
s = Solver()
```

```
s.add(Not(ULT(input_size, 2))) #  $\neg(\text{input\_size} < 2)$ 
s.add(result_size == input_size - 2) # result_size = input_size - 2
s.add(UGE(i, 1)) #  $i \geq 1$ 
s.add(ULT(i, input_size - 1)) #  $i < \text{input\_size} - 1$ 
```

```
s.add(
    Not(
        And(
            ULT(i - 1, result_size), #  $i - 1 < \text{result\_size}$ 
            ULT(i - 1, input_size), #  $i - 1 < \text{input\_size}$ 
            ULT(i, input_size), #  $i < \text{input\_size}$ 
            ULT(i + 1, input_size), #  $i + 1 < \text{input\_size}$ 
        )
    )
) # Prove no out-of-bounds reads
```

```
result = s.check()
print(result) # unsat
if result == sat: # A model was found that satisfies the assumptions
    print(s.model()) # Show model
elif result == unsat: # A contradiction was found
    print(s.proof()) # Show proof
```

```
std::vector<float> moving_average(const std::vector<float> &input) {
    if (input.size() < 2)
        return {};

    std::vector<float> result(input.size() - 2);

    for (size_t i = 1; i < input.size() - 1; ++i)
        result[i - 1] = (input[i - 1] + input[i] + input[i + 1]) / 3.;

    return result;
}
```

Prove: no divide by zero using C++26 contracts (not curriculum)

```
from z3 import * # type: ignore
```

```
set_param("proof", True)
```

```
r, b = BitVecs('r b', 32)
```

```
s = Solver()
```

```
s.add(r > 0)
```

```
s.add(b == r)
```

```
s.add(Not(b != 0)) # Prove divide precondition
```

```
result = s.check()
```

```
print(result) # unsat
```

```
if result == sat: # A model was found that satisfies the assumptions
```

```
    print(s.model()) # Show model
```

```
elif result == unsat: # A contradiction was found
```

```
    print(s.proof()) # Show proof
```

```
int positive_int()
    post (r: r > 0) {
        return 42;
    }

int divide(int a, int b)
    pre (b != 0) {
        return a / b;
    }

int main() {
    return divide(84, positive_int());
}
```